Number and Data Representation

Algorithms, Programming, and Data Representation · Module 92313 · Chapter 3

From Fundamentals to AI Apocalypse · personal study guide

Contents

Start from memory 2

How to use this guide 2

How the chapter fits together 3

3.1 Number systems and positional notation 4

3.2 Integer representations 6

3.3 Character data 8

3.4 Unsigned binary arithmetic 9

3.5 Two's-complement arithmetic and overflow 10

Formula map 11

Derive it cold 11

Explain it 12

Source check 12

Mixed practice 13

Exam-style problems 14

Self-check 15

Answers 16

This chapter shows how bit patterns encode numbers and characters, then uses those representations to perform arithmetic. The meaning of a bit pattern depends on the encoding and the number of bits.

Start from memory

Answer before reading. Mark each response sure, unsure, or guessed; uncertainty shows what to revisit.

1. What does the position of a digit mean in a base-b number?
2. What is the signed range of an n-bit two's-complement value?
3. How do you negate a two's-complement bit pattern?
4. How many binary digits correspond to one hexadecimal digit?
5. What does a discarded carry-out tell you about overflow?

Answers: see "Start from memory" under Answers at the end.

How to use this guide

Study one section, hide it, and reconstruct the rules and examples from memory. Then solve mixed questions before checking the worked answers.

1. **Day 0:** Try the cold start and study §3.1. Convert values both ways without a calculator.
2. **Day 1:** Rebuild the integer-encoding table in §3.2; solve practice 1–8.
3. **Day 3:** Recall the character encodings in §3.3 and work through unsigned arithmetic in §3.4.
4. **About day 7:** Re-derive two's-complement addition and subtraction in §3.5; solve practice 9–16.
5. **About day 14:** Complete both exam-style problems without notes and check the source typo.
6. **Before the exam:** Tick only skills you can demonstrate cold; revisit sections named in missed answers.

When stuck: write the bit width and encoding first. Show each conversion or arithmetic step, then check the answer against the allowed range.

How the chapter fits together

First learn how positional notation creates values. Then learn the encoding rules that assign values to bit patterns; finally apply arithmetic rules and check the fixed-width result.

Represent · map information to bits

- **Number bases** §3.1

  digits × powers of the base

  Rests on: positional notation, with the rightmost place worth one.

- **Integer encodings** §3.2

  one bit pattern, several interpretations

  Rests on: a fixed width and an explicit signed or unsigned encoding.

Encode · store other data

- **Character codes** §3.3

  character → code point → encoded bits

  Rests on: agreed mappings between symbols and numeric codes.

- **Unsigned arithmetic** §3.4

  carry · borrow · partial products

  Rests on: positional values and column-wise binary operations.

Compute · preserve meaning

- **Signed arithmetic** §3.5

  fixed width → decode → check overflow

  Rests on: two's-complement interpretation and its representable range.

Question	Useful method
What value does this bit pattern mean?	State width and encoding, then decode.
How do I convert a base-b number?	Expand by powers or use repeated division.
Did signed arithmetic overflow?	Check the exact result against the fixed-width range.

3.1 Number systems and positional notation

A numeral is a compact way to write a sum of place values. Binary, octal, decimal, and hexadecimal use the same positional rule with different bases.

Expand and convert

In base b, each digit d_i contributes $d_i b^i$, where the rightmost digit has i=0. Binary digits are 0 or 1. Decimal-to-binary repeated division records remainders; read them from last to first.

Positional value

$$(d_k \ldots d_1 d_0)_b = \sum d_i b^i$$

$$(1100010)_2 = 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^1 = 98$$

The worked binary number is 98 in decimal. Subscripts identify the base.

Worked conversion · decimal to binary

Convert 45 to binary by repeated division.

1. $45 \div 2$ gives quotient 22, remainder 1.
2. $22 \div 2$ gives quotient 11, remainder 0; then 11 gives 5 remainder 1.
3. Continue: 5 gives 2 remainder 1; 2 gives 1 remainder 0; 1 gives 0 remainder 1.
4. Read remainders upward: 101101_2. Check: 32+8+4+1=45. §3.1

Octal and hexadecimal

Because $8=2^3$, each octal digit corresponds to three binary digits. Because $16=2^4$, each hexadecimal digit corresponds to four bits. Group binary digits from the right and pad the leftmost group with zeroes if needed. In hexadecimal, A–F mean 10–15.

Compact binary groups

$$10101_2 = 010\,101 = 25_8$$
$$10010100_2 = 1001\,0100 = 94_{16}$$
$$1\,\text{byte} = 8\,\text{bits} = 2\,\text{hexadecimal digits}$$

The lecture uses 16-bit words and 32-bit long words as examples; the sizes named by these terms can depend on a computer architecture.

Trap: do not read a binary string as an ordinary decimal integer. Attach its base, expand it, and only then compare values.

Recall check · answer without looking

1. What power of the base multiplies the rightmost digit?
2. Why can binary be grouped in threes to convert to octal?
3. What is A in hexadecimal worth in decimal?

Answers: see "3.1 Number systems and positional notation" under Answers at the end.

3.2 Integer representations

A computer stores a fixed number of bits, not a plus or minus sign unless the encoding defines one. The same pattern can mean different values under different encodings.

Unsigned and signed ranges

An unsigned n-bit pattern represents 0 through 2^n-1. Signed encodings trade positive range for negative values. The source compares sign-and-magnitude, one's complement, two's complement, excess (biased) notation, and binary-coded decimal (BCD).

Ranges for n bits

$$\text{Unsigned}: 0 \dots 2^n - 1$$
$$\text{Sign and magnitude}: -(2^{n-1} - 1) \dots +(2^{n-1} - 1)$$
$$\text{One's complement}: -(2^{n-1} - 1) \dots +(2^{n-1} - 1)$$
$$\text{Two's complement}: -2^{n-1} \dots 2^{n-1} - 1$$

Sign-and-magnitude and one's complement have two zero encodings. Two's complement has one zero and one extra negative value.

Worked comparison · decode 1111 in four encodings

1. Unsigned: 15.
2. Sign-and-magnitude: sign bit 1 and magnitude 111, so -7.
3. One's complement: invert to 0000, so -0, which equals zero but has a distinct pattern.
4. Two's complement: the unsigned value 15 represents $15-16=-1$. §3.2

Two's complement and bias

To form $-x$ in n-bit two's complement, invert all n bits of x and add 1; discard a carry beyond the width. Equivalently, interpret a pattern with top bit 1 by subtracting 2^n from its unsigned value. In excess-B notation, actual value = stored unsigned value − B. The lecture's three-bit excess-4 example therefore maps 000 to -4 and 111 to 3.

Decode rules

$$\text{Two's}-\text{complement value} = \text{unsigned(pattern)} - 2^n \text{ when the top bit is 1}$$
$$\text{Negate x}: \text{invert n bits, then add 1}$$
$$\text{Excess}-\text{B value} = \text{stored value} - \text{B}$$

Keep the width fixed throughout. The same bit pattern has no signed meaning until an encoding is specified.

BCD stores decimal digits separately

In four-bit BCD, each decimal digit has its own four-bit code. For example, decimal 59 is 0101 1001 in BCD, while the ordinary binary value 59 is 00111011. BCD uses more bits but keeps each decimal digit directly readable.

Trap: do not convert all BCD bits as one binary number. Decode each four-bit group as a decimal digit.

Recall check · answer without looking

1. What is the four-bit two's-complement range?
2. Why is the two's-complement range asymmetric?
3. What is the BCD encoding of decimal 42?

Answers: see "3.2 Integer representations" under Answers at the end.

3.3 Character data

Numbers are not the only data stored as bits. A character encoding assigns numeric codes to letters, digits, punctuation, and control characters.

ASCII and Unicode

ASCII uses seven bits, giving 128 code values. The lecture describes eight-bit extensions that use the additional 128 patterns, but those upper values depend on the particular extension. Unicode assigns code points to characters across many writing systems; its first 128 code points match ASCII. A code point and its byte encoding are related but are not the same concept.

Character-code facts

$$7 - \text{bit ASCII} : 2^7 = 128 \,\text{code values}$$
$$\text{ASCII code points} : 0 \ldots 127$$

The first 128 Unicode code points correspond to ASCII. Byte encodings represent code points as bytes by their own rules.

Worked encoding · read a short ASCII string

The lecture's example encodes "Fred" in hexadecimal as 46 72 65 64.

1. Split the bytes: 46, 72, 65, 64.
2. Read each as a hexadecimal ASCII code: 46 is F, 72 is r, 65 is e, and 64 is d.
3. The four codes decode to "Fred". §3.3

Trap: an encoding is a mapping, not a property of the bit pattern by itself. Identify the character set and any byte encoding before decoding text.

Recall check · answer without looking

1. How many code values are available in seven-bit ASCII?
2. What is the relation between ASCII and the first 128 Unicode code points?

Answers: see "3.3 Character data" under Answers at the end.

3.4 Unsigned binary arithmetic

Binary arithmetic follows the same place-value logic as decimal arithmetic. Carries and borrows move between columns, while the fixed bit width limits which values can be stored.

Add and subtract

Add each column and carry 1 when a column totals 2. For subtraction, borrow from a higher column when the current minuend bit is too small; a borrowed binary 1 is worth 2 in the current column. Subtraction can also be checked by adding the difference to the subtrahend.

Worked addition · include the carry

Add 111011_2 and 101010_2.

1. From right to left: 1+0=1.
2. 1+1=10, so write 0 and carry 1. Continue each column with its carry.
3. The result is 1100101_2. Check: 59+42=101, and 1100101_2=101. §3.4

Multiply and divide

Binary multiplication uses partial products, like decimal long multiplication, but each multiplier bit is only 0 or 1. Binary division uses quotient and remainder. For divisor $d \neq 0$, dividend = quotient×divisor + remainder, where $0 \leq \text{remainder} < \text{divisor}$.

Division check

$$\text{dividend} = \text{quotient} \times \text{divisor} + \text{remainder}$$
$$0 \leq \text{remainder} < \text{divisor}$$

Substitute the quotient, divisor, and remainder to check a division result.

Memory hook: in binary addition, 1+1 writes 0 and carries 1. In subtraction, a borrow makes the current column worth 2 before subtracting.

Recall check · answer without looking

1. What bit do you write for 1+1, and what do you carry?
2. What inequality must the remainder satisfy?

Answers: see "3.4 Unsigned binary arithmetic" under Answers at the end.

3.5 Two's-complement arithmetic and overflow

Two's complement lets the same adder perform signed addition and subtraction. Correct bit operations still need a range check: a fixed-width result can wrap to a pattern with a different signed value.

Addition, subtraction, and overflow

Add the bit patterns and discard any carry beyond the most significant bit. For subtraction, negate the subtrahend and add it to the minuend. Signed overflow occurs when the exact mathematical result lies outside the representation's range. For addition, it occurs when two operands with the same sign produce a result with the opposite sign; operands of unlike signs cannot overflow.

n-bit two's-complement checks

$$\text{Range}: -2^{n-1} \ldots 2^{n-1} - 1$$

$$A - B = A + (\text{two's} - \text{complement negation of } B)$$

$$\text{Same} - \text{sign addition with opposite} - \text{sign result} \Rightarrow \text{overflow}$$

Discarding carry-out is part of fixed-width addition. Carry-out alone does not signal signed overflow.

Worked example · detect overflow

Use four-bit two's complement to calculate $-7+(-6)$.

1. Encode -7 as 1001 and -6 as 1010.
2. Add: 1001+1010=1 0011. Discard the carry-out, leaving 0011.
3. The stored result 0011 is $+3$, but the exact sum is -13, outside the four-bit range $-8\ldots+7$. Both inputs were negative and the result is positive, so overflow occurred. §3.5

Multiplication and division

The lecture notes that ordinary unsigned multiplication cannot simply be applied to signed two's-complement patterns and read at the original width. A full product may require up to twice as many bits as the operands; retain a sufficient width and interpret the result consistently. For a simple signed multiplication, multiply magnitudes and make the result negative exactly when the operand signs differ, then check the available width. The supplied slides introduce this idea but do not give a complete signed division algorithm.

Sign check: do not treat discarded carry as overflow. Decode the fixed-width result and compare it with the exact mathematical result and allowed range.

Recall check · answer without looking

1. How is A−B implemented in two's-complement arithmetic?
2. Can two's-complement addition overflow when the operands have unlike signs?
3. Why can a signed product need more bits than either input?

Answers: see "3.5 Two's-complement arithmetic and overflow" under Answers at the end.

Formula map

Idea	Relation	Use
Positional value	$\Sigma d_i b^i$	Convert a base-b numeral to a value.
Unsigned n-bit range	$0 \ldots 2^n-1$	Check if a non-negative integer fits.
Two's-complement range	$-2^{n-1} \ldots 2^{n-1}-1$	Check signed fixed-width values.
Two's-complement negation	Invert n bits, then add 1	Negate within the selected width.
Excess-B	actual = stored − B	Decode a biased pattern.
Division	dividend = quotient×divisor + remainder	Verify quotient and remainder.
Signed overflow	exact result outside representable range	Determine whether the encoded answer is valid.

Check every answer

State the base, encoding, and bit width

Verify place values or arithmetic in decimal

Compare signed results with the allowed range

Units are not used in these number-representation problems; check the encoding and range instead.

Derive it cold

Cover the explanations and rebuild each result on blank paper.

1. **Base conversion:** expand a binary numeral into powers of 2. §3.1
2. **Binary to octal/hex:** derive why groups of 3/4 bits match one digit. §3.1
3. **Two's-complement negation:** show why invert-and-add-one gives 2^n-x. §3.2
4. **Signed range:** determine the lowest and highest n-bit two's-complement values. §3.2
5. **Unsigned division:** verify a quotient by reconstructing the dividend. §3.4
6. **Addition overflow:** explain why same-sign inputs can produce an invalid opposite-sign result. §3.5
7. **Subtraction:** replace $A-B$ with $A+(-B)$ and show the bit width at every step. §3.5

Explain it

Answer each why or when question aloud before opening its model response.

1. Why does the rightmost binary digit have weight 1?
2. Why are octal and hexadecimal convenient ways to write binary?
3. Why do sign-and-magnitude and one's complement have two zero patterns?
4. Why does two's complement have one extra negative value?
5. When is carry-out discarded, and why is that not enough to detect signed overflow?
6. Why can BCD be easier for decimal display but less space-efficient?

Answers: see "Explain it" under Answers at the end.

Source check

The rendered slide table was checked visually; this is a transcription error, not a different encoding rule.

- **Slide 24 · missing minus sign in the Excess-8 row:** the entry for bit pattern 0111 is printed as 1. With four-bit excess-8, the decoded value is stored value 7 minus bias 8, which equals −1. The surrounding table and the lecture's earlier example use that mapping.

Mixed practice

Try all 20 problems before opening the worked answers. The topics are interleaved so you must select the encoding or operation each time.

1. Convert 37_{10} to binary and check by place value. §3.1
2. Decode 1110 as a four-bit two's-complement number. §3.2
3. How many bits represent one hexadecimal digit? §3.1
4. Encode decimal 73 in four-bit BCD. §3.2
5. How many values are available in seven-bit ASCII? §3.3
6. Add 1011_2 and 0110_2 as unsigned values. §3.4
7. What is the four-bit two's-complement range? §3.2
8. Convert 101101_2 to octal. §3.1
9. In excess-4, what value does the three-bit pattern 110 represent? §3.2
10. For $29 \div 5$, give quotient and remainder and verify the division identity. §3.4
11. Negate 0011 in four-bit two's complement. §3.2
12. Convert 11101010_2 to hexadecimal. §3.1
13. Why does two's-complement addition discard a carry-out? §3.5
14. Write decimal 59 in BCD and ordinary binary. §3.2
15. Compute $7-(-6)$ using four-bit two's complement. State the exact result and whether it fits. §3.5
16. What do the first 128 Unicode code points have in common with ASCII? §3.3
17. Multiply 101_2 by 11_2 as unsigned values. §3.4
18. In four-bit two's complement, add 0101 and 0011. Does it overflow? §3.5
19. What is the decoded value of pattern 0111 in four-bit excess-8? §3.2
20. What is one reason BCD can be useful despite its extra storage? §3.2

Answers: see "Mixed practice" under Answers at the end.

Exam-style problems

Allow about 20 minutes per problem. State the bit width and encoding before each calculation.

1 · Compare encodings

Use four bits to represent the integer -5. Compare sign-and-magnitude, one's complement, and two's complement.

1. Write the four-bit pattern in each encoding.
2. State the range and number of zero patterns for each representation.
3. Explain why two's complement is useful for arithmetic despite its asymmetric range.

Answers: see "Exam-style problems" under Answers at the end.

2 · Convert, calculate, and check

Use four-bit two's-complement inputs for -5 and $+3$.

1. Write both input patterns and add them, showing any carry-out.
2. Decode the retained four-bit result and compare it with the exact sum.
3. State whether overflow occurred and explain why carry-out is not the test.

Answers: see "Exam-style problems" under Answers at the end.

Self-check

Tick a skill only in a later session, at least one day after studying, when you can demonstrate it without notes.

- ☐ Expand numbers by powers of their base. (1, 8, 12)
- ☐ Convert binary groups to octal and hexadecimal. (3, 8, 12)
- ☐ State signed and unsigned ranges from a bit width. (7)
- ☐ Decode patterns using distinct signed encodings. (2, 19)
- ☐ Negate a two's-complement value at fixed width. (11)
- ☐ Decode an excess-B value. (9, 19)
- ☐ Write and read decimal digits in BCD. (4, 14, 20)
- ☐ Explain ASCII and Unicode code-point relationships. (5, 16)
- ☐ Add binary values and carry correctly. (6, 17)
- ☐ Verify quotient and remainder. (10)
- ☐ Add signed values and detect overflow. (13, 18)
- ☐ Subtract by negating and adding. (15)
- ☐ Identify and correct the slide 24 excess-8 typo. (19)
- ☐ Rebuild the derivations from the formula list.
- ☐ Complete both exam-style problems with checks.

Answers

Start from memory

1. It is the digit's coefficient on a power of the base, counting from power zero at the right. §3.1
2. -2^{n-1} through $2^{n-1}-1$. §3.2
3. Invert all bits and add 1, keeping the chosen width. §3.2
4. Four bits, because $16 = 2^4$. §3.1
5. Nothing by itself. In fixed-width two's-complement addition, carry-out is discarded and is not the signed-overflow test. §3.5

3.1 Number systems and positional notation

1. The zeroth power, which is 1. §3.1
2. Each octal digit represents 0–7, exactly the range of three bits, and $8=2^3$. §3.1
3. 10. §3.1

3.2 Integer representations

1. -8 through $+7$. §3.2
2. There is one zero and one additional negative pattern, 1000, representing -8. §3.2
3. 0100 0010, one four-bit code for 4 and one for 2. §3.2

3.3 Character data

1. $2^7=128$. §3.3
2. The Unicode code points 0–127 correspond to the ASCII characters; their byte representation depends on the encoding used. §3.3

3.4 Unsigned binary arithmetic

1. Write 0 and carry 1 to the next column. §3.4
2. For a positive divisor, $0 \le \text{remainder} < \text{divisor}$. §3.4

3.5 Two's-complement arithmetic and overflow

1. Negate B in the chosen width, then add it to A and discard carry-out. §3.5
2. No. Overflow in addition requires two operands with the same sign and a result of the opposite sign. §3.5
3. The product magnitude can exceed the range of either input width; a full-width product may need up to twice as many bits. §3.5

Explain it

1. The rightmost place is position zero, and any base raised to zero is 1. Its bit contributes either zero or one to the value.
2. Each octal digit maps to exactly three bits and each hexadecimal digit to four. Grouping bits preserves their value while shortening the written form.
3. Both encodings use a sign bit and a separate magnitude/complement pattern for zero. That creates distinct positive-zero and negative-zero bit patterns.
4. With n bits, one pattern is used for zero and the remaining patterns cannot be divided evenly into equal positive and negative ranges. Two's complement assigns the extra pattern to the negative side.
5. Fixed-width addition retains only n result bits, so carry beyond the word is discarded. Signed overflow depends on the signs of the inputs and result, not on that discarded bit.
6. Each four-bit BCD group can be read as one decimal digit, but four bits are used per decimal digit even when ordinary binary could represent the whole number with fewer bits.

Mixed practice

1. $37=32+4+1$, so 100101_2. §3.1
2. Unsigned value $14-16=-2$. §3.2
3. Four bits, since $16=2^4$. §3.1
4. 7 is 0111 and 3 is 0011, so 73 is 0111 0011 in BCD. §3.2
5. $2^7=128$ code values. §3.3
6. $1011+0110=10001$. As an unsigned five-bit result it is 17. §3.4
7. -8 through $+7$. §3.2
8. Group 101 101; each group is 5, so 55_8. §3.1
9. Pattern 110 is 6; $6-4=2$. §3.2
10. $29=5\times5+4$, so quotient 5 and remainder 4; $0\leq4<5$. §3.4
11. Invert 0011 to 1100 and add 1: 1101, which is -3. §3.2
12. 1110 1010 maps to E A, so EA_{16}. §3.1
13. The machine keeps only the fixed n result bits. Carry-out is outside that word and does not alone indicate signed overflow. §3.5
14. BCD: 0101 1001. Ordinary binary: 111011. §3.2
15. -6 is 1010; invert and add 1 to get $+6=0110$. Then $0111+0110=1101$, which decodes as -3. The exact result is $+13$, outside $-8\ldots+7$, so overflow occurs. §3.5
16. Unicode code points 0–127 correspond to ASCII characters. §3.3
17. 101 is 5 and 11 is 3; $5\times3=15=1111_2$. §3.4
18. $0101+0011=1000$. The exact result is $+8$, outside the four-bit range $-8\ldots+7$; the two positive inputs produced a negative-sign pattern, so it overflows. §3.5
19. $7-8=-1$. §3.2
20. BCD keeps each decimal digit directly encoded, which can simplify decimal display or input/output. §3.2

Exam-style problems

1. $+5$ is 0101. Sign-and-magnitude is 1101. One's complement is 1010, the inversion of 0101. Two's complement is 1011, the inversion plus 1. §3.2
2. For four bits, sign-and-magnitude and one's complement both range from -7 to $+7$ and have two zero patterns. Two's complement ranges from -8 to $+7$ and has one zero. §3.2
3. Negation and subtraction use the same fixed-width adder: invert the subtrahend and add one. Its one extra negative value is usually a small trade-off for simpler arithmetic hardware. §3.2 §3.5

Exam-style problems

1. -5 is 1011 and $+3$ is 0011. Addition gives 1110 with no carry-out. §3.5
2. 1110 is 14 unsigned; subtract 16 to decode -2. The exact sum is also -2. §3.2 §3.5
3. No overflow: the exact result -2 lies in $-8\ldots+7$. Carry-out is not the signed-overflow criterion; compare the signs or the exact result to the range. §3.5

Personal study guide based on the supplied "03_data_representation.pdf" (41 slides; slide 40 is dated 22 April 2024). Written in my own words; no lecture pages or figures are reproduced. The topic headings in this guide organise the slide sequence. Not an official FAU publication; check course details against the current notes.